

Therac-25 and Risks: Software Quality Assurance Review

Zixuan Liang

Harrisburg University of Science and Technology

CISC 594: Software Testing Principles and Techniques

Dr. Khalid Lateef

June 24, 2026

Therac-25 and Risks: Software Quality Assurance Review

Introduction

The Therac-25 is one of the most important case studies in software quality assurance because it shows how software defects become dangerous when they are embedded inside a larger safety-critical system. The machine was not merely a buggy application. It was a medical radiation device whose software was trusted to control high-energy treatment beams, interpret operator input, verify machine setup, and recover from errors. Between June 1985 and January 1987, six known patients received massive radiation overdoses from Therac-25 machines, leading to severe injury and death (Leveson & Turner, 1993). The case therefore belongs in an SQA course because the failures were not limited to coding. They included weak requirements, poor hazard analysis, inadequate software testing, overconfidence in reused software, missing independent safety interlocks, weak documentation, poor incident reporting, and inadequate communication with users and regulators.

This updated review answers the assignment questions by identifying the Therac-25, explaining its purpose, describing when it was built, identifying its predecessors, summarizing three source documents, and then reviewing the case from a software quality assurance and risk-management perspective. It also adds answers to Questions 6, 7, and 8 by identifying the top risks, scoring them by likelihood and impact, and recommending software and hardware controls.

What the System Was

Therac-25 was a computer-controlled medical linear accelerator, or linac, manufactured by Atomic Energy of Canada Limited. A linac accelerates electrons to create therapeutic radiation beams used in cancer treatment. The Therac-25 could operate in two treatment modes.

In electron mode, it delivered electron beams for relatively shallow tissue treatment. In photon, or X-ray, mode, it converted high-energy electrons into X-rays for deeper tissue treatment (Leveson & Turner, 1993). The system was controlled by a DEC PDP-11 computer, and the software handled patient prescription entry, machine setup, beam activation, monitoring, and error messages.

The machine mattered technically because it represented a stronger shift toward software-controlled safety than its predecessors. Earlier machines used computer control largely as an added convenience on top of hardware that could stand alone. Therac-25 was designed from the beginning to depend more heavily on computer control. AECL did not duplicate all of the older hardware safety mechanisms, which meant the software became a central part of the safety case rather than a supporting convenience (Leveson & Turner, 1993).

Purpose of the System

The purpose of Therac-25 was to treat cancer patients by delivering prescribed radiation doses to tumors while limiting damage to surrounding healthy tissue. In radiation therapy, dose, beam type, beam energy, beam shape, patient position, and machine configuration must agree with the physician's treatment plan. The value proposition of Therac-25 was that a single compact machine could support both electron and photon therapy, making it more versatile and economical than separate single-purpose machines. That same dual-mode flexibility, however, increased the need for strong setup verification because the machine's target, filters, collimators, dose monitoring, scanning magnets, and beam settings had to match the selected treatment mode.

When It Was Built

The development timeline began before the commercial machine appeared. AECL and the French company CGR collaborated on earlier Therac machines in the early 1970s. AECL

then developed the double-pass accelerator concept in the mid-1970s, produced a first hardwired Therac-25 prototype in 1976, and made the completely computerized commercial version available in late 1982 (Leveson & Turner, 1993). Eleven Therac-25 units were installed in the United States and Canada. The known overdose accidents occurred from 1985 through 1987, and the machine was recalled in 1987 for extensive changes.

Date or Period	Therac-25 Context
Early 1970s	AECL and CGR collaborated on earlier Therac linear accelerators.
Mid-1970s	AECL developed the double-pass accelerator concept that made a compact dual-mode machine possible.
1976	AECL produced the first hardwired Therac-25 prototype.
Late 1982	The completely computerized commercial Therac-25 became available.
1985-1987	Six known massive overdose accidents occurred in the United States and Canada.
1987 onward	The machine was recalled for extensive hardware, software, documentation, and safety-design changes.

Predecessors of Therac-25

The main predecessors were Therac-6 and Therac-20. Therac-6 was a 6 MeV accelerator that produced X-rays only. Therac-20 was a 20 MeV dual-mode accelerator that could deliver either X-rays or electrons. Both were based on older CGR machines and were augmented with PDP-11 computer control. The essential safety distinction is that Therac-6 and Therac-20 retained industry-standard hardware safety features and interlocks. The computer improved convenience, but the older machines still had independent protective mechanisms.

Therac-25 differed because it relied more heavily on software to maintain safe operation. Leveson and Turner (1993) explain that the Therac-20 had independent protective circuits and mechanical interlocks, while Therac-25 placed more safety responsibility on software. Related software problems were later found in Therac-20, but they did not injure patients because the hardware interlocks prevented the unsafe beam condition from reaching the patient. This comparison is one of the clearest SQA lessons in the case: software reuse does not transfer safety automatically, because safety is a property of the whole system in its operating context.

Source Document Summaries

Introductory Document: Online Ethics Center References

The Online Ethics Center source is a student-friendly introductory and bibliographic document for the Therac-25 case. It identifies Leveson and Turner's IEEE Computer article as the classical report on the system and explains that the case is useful not only as a technical failure but also as an ethics and professional responsibility case (Huff, 2003). Its value for this assignment is that it helps place the incident in a broader learning context. The document points readers toward accident investigation, system safety, human error, software ethics, and user responsibility sources. It also reinforces that Therac-25 is not only a story about a single race

condition. It is a case about professional judgment, manufacturer responsibility, disclosure, and the consequences of weak safety culture.

IEEE Research Paper 1: Leveson and Turner (1993)

Leveson and Turner's article, 'An Investigation of the Therac-25 Accidents,' is the central technical source. It provides the history of the machine, the accident chronology, the technical mechanism behind key overdoses, and the responses by AECL, users, and regulators. The paper argues that most accidents in complex systems are system accidents rather than single-cause events. That framing is crucial: the overdoses involved software defects, but they were also enabled by removed hardware interlocks, unrealistic risk assessment, weak documentation, inadequate test planning, lack of audit trails, poor error messages, and slow incident communication (Leveson & Turner, 1993).

The most important SQA contribution of the paper is its explanation that system testing alone was not enough. The Therac-25 software had concurrent tasks, shared variables, timing-sensitive behavior, and operator editing sequences that could produce unsafe states. Yet the available evidence suggested minimal unit-level software testing and no adequate software test plan. The paper also warns against overconfidence in reused code. Some routines came from earlier Therac software, but the new machine's hardware context removed safeguards that had previously hidden or mitigated software faults. In modern terms, the reused component changed assurance context, so its safety argument had to be reestablished from scratch.

IEEE Research Paper 2: Silvis-Cividjian (2024)

Silvis-Cividjian's 2024 IEEE Computer article revisits Therac-25 through interviews with Fritz Hager, the medical physicist who investigated the two Tyler, Texas accidents. This paper adds a human and historical layer to the canonical account. It describes the real machine,

reconstructs the gantry and user interface details, and explains the Malfunction 54 scenario from the perspective of someone who had to reproduce and measure the overdose condition (Silvis-Cividjian, 2024). The article is especially useful because it shows how confusing feedback and weak observability made diagnosis difficult for clinical users.

For SQA, the 2024 article emphasizes three enduring lessons: quality assurance, safety science, and human factors. It argues that software engineers working in mission-critical systems need domain-specific testing strategies, hazard analysis that considers interactions among hardware, software, and users, and error messages that help operators make safe decisions under stress. The article also shows the emotional burden placed on clinicians who trusted the system. That point matters because SQA failures in medical systems create harm beyond the immediate technical failure; they affect patients, families, operators, physicists, manufacturers, and public trust.

Software Quality Assurance Review

From an SQA perspective, the first major failure was inadequate hazard analysis. The Therac-25 safety analysis apparently excluded software design errors and focused on random hardware failures. That assumption was backwards for a machine whose safety increasingly depended on software. A realistic hazard analysis should have asked what unsafe control actions could occur if software state, operator input, and mechanical configuration diverged. It also should have treated the removal of independent hardware interlocks as a high-severity architectural change requiring explicit justification.

The second failure was inadequate requirements and safety constraints. A safe radiation therapy machine should have had requirements such as: the beam must not activate unless independent hardware and software checks agree on mode, energy, turntable position, scanning

magnet state, and dose monitor readiness; severe malfunctions must suspend treatment rather than allow repeated proceed commands; and operators must receive error messages that distinguish underdose, overdose, and unsafe machine state. The actual system allowed treatment pauses and cryptic codes such as Malfunction 54, which did not tell operators that a potentially catastrophic overdose had occurred.

The third failure was weak software design and verification. The accidents involved timing-sensitive interactions among concurrent tasks and shared variables. A fast operator could edit treatment parameters during a narrow time window, leaving the software's treatment setup inconsistent with the operator-visible screen. That kind of defect is hard to find with ordinary happy-path system tests. It calls for module-level tests, stress tests around editing timing, concurrency analysis, formal review of shared-state assumptions, fault injection, and tests derived from identified hazards.

The fourth failure was overreliance on software without independent safety barriers. Therac-20 reportedly had a similar software problem, but its hardware interlocks prevented patient injury. Therac-25 removed or failed to duplicate several of those protections. The result was a single point of catastrophic failure: if the software believed the setup was safe, the machine could deliver an unsafe beam. Safety-critical design should assume software can fail and should prevent one software defect from directly becoming patient harm.

The fifth failure was poor incident response and configuration control. After early accidents, AECL and users did not have enough shared information, audit trails, or transparent reporting to identify the pattern quickly. Fixes were initially narrow, such as disabling an editing key, rather than system-level corrections addressing root causes. Good SQA requires a closed-loop corrective action process: collect incident data, reproduce the fault, analyze root causes,

update hazards and requirements, modify design, regression test the fix, communicate risk clearly to all users, and monitor for recurrence.

A modern SQA response would combine traditional verification with system-safety assurance. Requirements would be traceable to hazards. Safety-critical software would receive independent review. Test design would include boundary values, race conditions, recovery paths, and abnormal operator workflows. The system would record audit logs sufficient to reconstruct unsafe states. User-interface requirements would treat clarity and fail-safe behavior as safety features, not cosmetic details. Most importantly, software would not be the only barrier between a timing defect and a fatal radiation beam.

Risk Assessment Method

For Questions 6 and 7, the risks are scored using a 1-5 likelihood scale and a 1-5 impact scale. The risk score is calculated as likelihood multiplied by impact. A score near 25 represents a top-priority risk because it is both likely enough to be credible and severe enough to demand immediate control. In the Therac-25 case, impact is weighted heavily because the system was safety-critical: a single failure could injure or kill a patient. The complete class-format risk worksheet is provided separately as *Therac25_Risk_Table_Zixuan_Liang.xlsx*.

Q6 and Q7: Top Risks and Risk Scores

The top five risks associated with Therac-25 are listed in Table 2. These risks are not independent; they form a chain. Weak hazard analysis and testing allowed unsafe software behavior to remain in the product. Missing independent hardware barriers allowed the software defect to reach the patient. Poor operator feedback and weak incident reporting then delayed recognition and correction across the installed base.

Table 2

Therac-25 Top Risk Scores

Rank	Risk	Likelihood	Impact	Risk score
1	Unsafe radiation overdose from software-controlled mode mismatch	5	5	$5 \times 5 = 25$
2	Removal or absence of independent hardware interlocks	4	5	$4 \times 5 = 20$
3	Inadequate software testing and hazard analysis	4	5	$4 \times 5 = 20$
4	Cryptic operator feedback and unsafe recovery behavior	4	4	$4 \times 4 = 16$
5	Weak incident reporting, audit trails, and corrective action	3	5	$3 \times 5 = 15$

Risk Justification

The highest-ranked risk is unsafe radiation overdose from a software-controlled mode mismatch. This receives likelihood 5 and impact 5 because the Malfunction 54 sequence was reproducible and because the clinical consequence was catastrophic. Leveson and Turner (1993) show that the problem was not merely theoretical; multiple patients were actually injured or killed. In a medical device, confirmed reproducibility plus fatal consequence makes this the dominant risk.

The second risk is the removal or absence of independent hardware interlocks. This receives likelihood 4 and impact 5 because it was embedded in the design of Therac-25 rather than a single random defect. Earlier machines retained hardware interlocks that could prevent software errors from injuring patients. Therac-25 placed more responsibility on software, so an unsafe software state could become an unsafe physical beam.

The third risk is inadequate software testing and hazard analysis. This receives likelihood 4 and impact 5 because the safety analysis apparently did not treat software design errors as a central hazard, even though software controlled critical functions. The impact is maximal because insufficient testing of safety-critical state transitions, timing windows, and concurrency can allow lethal defects to escape into clinical use.

The fourth risk is cryptic operator feedback and unsafe recovery behavior. This receives likelihood 4 and impact 4 because operators encountered frequent error messages and could become habituated to them. The phrase Malfunction 54 did not clearly communicate that a severe overdose condition might have occurred. Allowing repeated proceed commands after serious malfunctions increased operational exposure.

The fifth risk is weak incident reporting, audit trails, and corrective action. This receives likelihood 3 and impact 5. The exact accident sequence was rare, but the organizational weakness was severe because incomplete logs, slow pattern recognition, and insufficient field communication allowed hazards to remain active across multiple clinical sites.

Q8: Software and Hardware Solutions

The first set of controls should prevent an unsafe beam from being physically possible. Therac-25 needed independent hardware interlocks that verified turntable position, beam mode, beam energy, scanning magnet state, target/filter position, and dose-rate behavior before

enabling radiation. If the hardware state and software-selected mode did not agree, the beam-enable path should have failed closed. Independent dose monitoring should also have been capable of detecting the high-current unscanned beam condition instead of saturating and reporting a misleading low dose.

The second set of controls should improve software design and verification. The control software should have been redesigned around explicit safety states, with atomic transitions between prescription entry, setup verification, beam ready, treatment, pause, and abort. Shared-variable concurrency should have been reviewed formally, and rapid editing paths should have been tested through stress tests and race-condition tests. Requirements should have been traced to hazards, and every hazard should have had corresponding design controls and regression tests.

The third set of controls should improve the user interface and recovery behavior. Severe faults should produce plain-language messages that identify the safety implication, not only a numeric malfunction code. A serious dose or mode inconsistency should lock out treatment until a qualified physicist or service engineer investigates. The system should not allow operators to repeatedly resume treatment after severe malfunctions, because recovery behavior is itself a safety-critical requirement.

The fourth set of controls should improve lifecycle SQA. The machine should record audit logs sufficient to reconstruct operator input, software state, hardware sensor readings, beam mode, dose monitor readings, and error transitions. AECL should have maintained a formal hazard log, corrective-action process, and field safety notification procedure. Once a serious unexplained injury occurred, all users and regulators should have received timely warnings, interim operating restrictions, and validated corrective actions.

The practical lesson is that no single control would have been enough. The right solution combines hardware defense in depth, safer software architecture, hazard-driven testing, clearer human-machine interaction, and disciplined incident response. That layered approach is the core of software quality assurance for safety-critical systems.

Conclusion

Therac-25 was a sophisticated medical radiation therapy system built to help cancer patients, but its accidents show what happens when software quality assurance is treated as ordinary testing rather than safety assurance. The machine's purpose was medically valuable, and its technical design was innovative. The tragedy was that innovation outpaced the assurance practices needed to control it. The risk assessment reinforces the same point: the top risks were not isolated bugs but connected failures in architecture, verification, operator communication, and lifecycle response. Safety-critical systems require explicit safety requirements, independent barriers, disciplined software engineering, transparent incident reporting, and verification strategies that match the risk of the system.

References

Huff, C. (2003). *References for Therac-25 case*. Online Ethics Center.

<https://doi.org/10.18130/mp9b-1467>

Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. *Computer*,

26(7), 18-41. <https://doi.org/10.1109/MC.1993.274940>

Silvis-Cividjian, N. (2024). Therac-25 accidents: We keep on learning from them. *Computer*,

57(12), 69-78. <https://doi.org/10.1109/MC.2024.3450197>

Appendix A

Source Documents Collected for Submission

The following source documents have been saved in the project folder under Therac25_sources and should be submitted with the assignment writeup if the LMS permits multiple uploads.

Leveson_Turner_1993_Investigation_Therac25.pdf

Silvis_Cividjian_2024_Therac25_Accidents.pdf

Online_Ethics_Therac25_References.html